Design and Implementation of a 5-Stage Pipelined RISC-V Processor

Introduction to Processor Architecture
Spring 2026

Contents

1 Introduction **4**
- 1.1 Team Members . . . 4
- 1.2 Project Objective . . . 5
- 1.3 Supported Instruction Set . . . 5

2 Five-Stage Pipeline Architecture **6**
- 2.1 Instruction Fetch Stage (IF) . . . 7
- 2.2 Instruction Decode Stage (ID) . . . 7
- 2.3 Execute Stage (EX) . . . 8
- 2.4 Memory Access Stage (MEM) . . . 8
- 2.5 Write Back Stage (WB) . . . 8
- 2.6 Pipeline Execution Example . . . 9

3 Overall Pipeline Datapath Design **10**

4 Pipeline Registers **10**
- 4.1 IF/ID Pipeline Register . . . 10
  - 4.1.1 Inputs . . . 10
  - 4.1.2 Outputs . . . 11
  - 4.1.3 Operation . . . 11
- 4.2 ID/EX Pipeline Register . . . 11
  - 4.2.1 Inputs . . . 12
  - 4.2.2 Outputs . . . 13
  - 4.2.3 Operation . . . 13

4.3 EX/MEM Pipeline Register 13
4.3.1 Inputs 14
4.3.2 Outputs 14
4.3.3 Operation 15
4.4 MEM/WB Pipeline Register 15
4.4.1 Inputs 15
4.4.2 Outputs 16
4.4.3 Operation 16

5 Hazard Handling Mechanisms 16
5.1 Pipeline Hazards Overview 16
5.2 Data Forwarding Unit 18
5.2.1 Inputs 18
5.2.2 Outputs 18
5.2.3 Forwarding Logic 19
5.3 Hazard Detection Unit 19
5.3.1 Inputs 20
5.3.2 Register Usage Detection 20
5.3.3 Hazard Condition 20
5.3.4 Pipeline Response 21
5.4 Control Hazards 21
5.4.1 Branch Resolution 21
5.4.2 Program Counter Update 21
5.4.3 Pipeline Flush Mechanism 22

6 Processor Implementation 22
6.1 Instruction Fetch Stage 22
6.2 Instruction Decode Stage 23
6.3 Execute Stage 23
6.4 Memory Access Stage 24
6.5 Write Back Stage 24
6.6 Hazard Handling Integration 24
6.7 Pipeline Integration 24
6.8 Clock Cycle Methodology 25
6.8.1 Positive Edge Triggered Pipeline Registers 25
6.8.2 Combinational Logic Between Clock Edges 25
6.8.3 Negative Edge Usage 26
6.8.4 Pipeline Progression 26

6.8.5 Reset Behavior 27

7 Verification and Testing **27**
7.1 Testbench Implementation 27
7.1.1 Processor Instantiation 27
7.1.2 Clock Generation 28
7.1.3 Reset Initialization 28
7.1.4 Instruction Loading 28
7.1.5 Simulation Control 28
7.1.6 Waveform Generation 29
7.1.7 Result Logging 29
7.2 Test Cases and Expected Results 29
7.2.1 Test Case 1: Arithmetic Operations 30
7.2.2 Test Case 2: Forwarding from EX Stage 30
7.2.3 Test Case 3: Forwarding from MEM/WB Stage 31
7.2.4 Test Case 4: Load-Use Hazard 31
7.2.5 Test Case 5: Branch Not Taken 32
7.2.6 Test Case 6: Branch Taken 32
7.3 Waveform Analysis 33
7.3.1 Test Case 1: Arithmetic Operations 33
7.3.2 Test Case 2: EX Stage Forwarding 34

8 Team Contributions **34**

1 Introduction

1.1 Team Members

- **Subal Manchanda (2024102033)**
- **Anish Toshniwal (2024102009)**
- **Parth Desai (2024102049)**

Modern processors are designed to achieve high performance by exploiting different forms of parallelism. One of the most widely used techniques to improve processor performance is **instruction pipelining**. In a sequential processor implementation, each instruction completes all stages of execution before the next instruction begins. As a result, only one instruction is processed at a time, which limits the overall throughput of the processor.

Processor performance is typically determined by three key factors: instruction count, clock cycle time, and the number of clock cycles per instruction (CPI). While the instruction set architecture and compiler influence the instruction count, the microarchitectural implementation of the processor determines both the clock cycle time and CPI [1]. A sequential implementation often results in a high CPI because each instruction must complete all stages of execution within a single long clock cycle.

To address this limitation, pipelining divides instruction execution into multiple stages and allows different instructions to be processed concurrently in different stages of the pipeline. This technique increases instruction throughput by enabling the processor to begin executing a new instruction before the previous instruction has completed execution.

In this project, a **5-stage pipelined RISC-V processor** is designed and implemented. The pipelined design is built upon the previously implemented sequential processor. The datapath is divided into five stages, each responsible for a specific part of instruction execution:

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory Access (MEM)

5. Write Back (WB)

Each stage performs a specific operation in the instruction execution process. By separating these operations into distinct stages and inserting pipeline registers between them, multiple instructions can be processed simultaneously at different stages of execution.

Although pipelining improves throughput, it also introduces several challenges. Since multiple instructions are executed concurrently, dependencies between instructions may lead to **pipeline hazards**. These hazards can cause incorrect program execution if not handled properly. Therefore, mechanisms such as **data forwarding**, **hazard detection**, and **control hazard handling** must be incorporated into the processor design.

In this project, the pipelined processor implements hazard handling mechanisms to ensure correct execution of instructions while maintaining high throughput. A forwarding unit is implemented to resolve data hazards by bypassing results between pipeline stages. A hazard detection unit is used to detect situations where forwarding alone is insufficient and stalls must be inserted into the pipeline. Additionally, control hazards caused by branch instructions are handled using a static branch prediction scheme.

The pipelined processor implemented in this project supports the same subset of the RISC-V instruction set as the sequential processor, including arithmetic operations, memory access instructions, and branch instructions. The design follows the classical RISC pipeline architecture described in modern processor design literature [1].

The remainder of this report describes the design and implementation of the pipelined processor, including the pipeline architecture, datapath organization, hazard handling mechanisms, and verification of the processor using test cases and waveform analysis.

1.2 Project Objective

The goal is to convert the sequential processor into a 5-stage pipelined RISC-V processor.

1.3 Supported Instruction Set

- add
- sub

- or
- and
- addi
- ld
- sd
- beq

2 Five-Stage Pipeline Architecture

To improve processor performance, modern processors exploit **instruction-level parallelism** through a technique known as **pipelining**. In a sequential processor, each instruction completes all stages of execution before the next instruction begins. As a result, only one instruction is processed at a time, leading to low throughput.

Pipelining improves performance by dividing the execution of an instruction into multiple stages and allowing different instructions to occupy different stages simultaneously. This enables the processor to start executing a new instruction every clock cycle once the pipeline is filled.

The processor implemented in this project follows the classical **five-stage RISC pipeline architecture**, consisting of:

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory Access (MEM)
5. Write Back (WB)

This design follows the classical RISC processor implementation described in Chapter 4 of the textbook, where the execution of instructions is decomposed into multiple stages corresponding to instruction fetch, operand read, computation, memory access, and register write-back operations [1].

Each stage performs a specific function in the execution of an instruction, and **pipeline registers** are inserted between stages to store intermediate data and control signals.

2.1 Instruction Fetch Stage (IF)

The Instruction Fetch stage is responsible for retrieving the next instruction from the instruction memory. The processor maintains a **Program Counter (PC)**, which stores the address of the instruction currently being executed.

During this stage, the following operations occur:

1. The Program Counter supplies the address to the instruction memory.
2. The instruction stored at that address is fetched.
3. The PC is incremented by 4 to point to the next sequential instruction.

Since RISC-V instructions are 32 bits (4 bytes) wide, the PC increments by 4 after each instruction fetch. The fetched instruction is then stored in the **IF/ID pipeline register** so that it can be used by the next stage.

According to the processor design described in the textbook, all instructions begin execution by using the PC to supply the address to the instruction memory and fetch the instruction [1].

2.2 Instruction Decode Stage (ID)

The Instruction Decode stage interprets the fetched instruction and prepares the operands required for execution. This stage performs several important tasks.

First, the instruction fields are decoded. The opcode determines the type of instruction being executed, while other fields specify the source registers, destination register, and immediate values.

Second, the register file is accessed to read the values of the source registers. The register file contains the processor's general-purpose registers and allows simultaneous reading of two registers and writing to one register.

Third, if the instruction contains an immediate operand, the **Immediate Generation Unit** extracts the immediate field and sign-extends it to the appropriate width.

Finally, the control unit generates the necessary control signals that determine the behavior of later stages of the pipeline.

All data and control signals generated during this stage are stored in the **ID/EX pipeline register**.

2.3 Execute Stage (EX)

The Execute stage performs arithmetic or logical operations using the **Arithmetic Logic Unit (ALU)**.

The ALU takes two input operands and performs operations such as addition, subtraction, logical AND, or logical OR depending on the instruction type. These operations are controlled by the ALU control signals generated during the decode stage.

For memory instructions, the ALU computes the effective memory address by adding the base register value and the sign-extended immediate.

For branch instructions, the ALU compares the values of two registers to determine whether the branch condition is satisfied.

In addition, the branch target address is calculated by adding the sign-extended branch offset to the current PC value. The result of these computations is stored in the **EX/MEM pipeline register**.

2.4 Memory Access Stage (MEM)

The Memory Access stage interacts with the data memory.

If the instruction is a load instruction, the processor reads data from the memory address computed in the execute stage. The data retrieved from memory will later be written back to the register file.

If the instruction is a store instruction, the processor writes data from the register file into the specified memory address.

For instructions that do not involve memory operations, this stage simply passes the ALU result forward without accessing memory.

The outputs of this stage are stored in the **MEM/WB pipeline register**.

2.5 Write Back Stage (WB)

The Write Back stage is the final stage of instruction execution. In this stage, the processor writes the result of the instruction back into the register file.

Depending on the instruction type, the value written back may come from one of two sources:

- The ALU result (for arithmetic or logical instructions)
- The data memory output (for load instructions)

A multiplexer controlled by the `MemToReg` signal selects the appropriate value to write into the destination register.

If the `RegWrite` control signal is asserted, the selected value is written into the destination register specified by the instruction.

Once the write-back operation is complete, the instruction has finished execution.

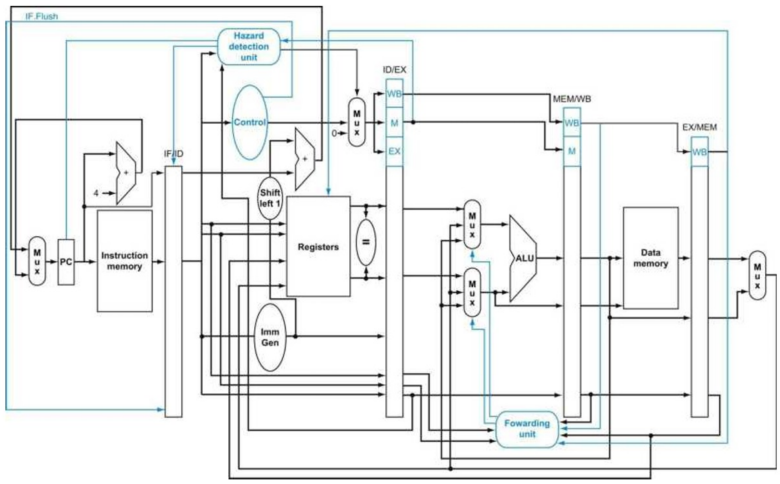
2.6 Pipeline Execution Example

The advantage of pipelining can be illustrated using a sequence of instructions. While one instruction is being executed in the execute stage, another instruction can be decoded, and a third instruction can be fetched simultaneously.

Cycle	IF	ID	EX	MEM	WB
1	I1				
2	I2	I1			
3	I3	I2	I1		
4	I4	I3	I2	I1	
5		I4	I3	I2	I1

Once the pipeline is filled, the processor completes approximately one instruction per clock cycle, significantly improving instruction throughput compared to a sequential processor.

3 Overall Pipeline Datapath Design



4 Pipeline Registers

4.1 IF/ID Pipeline Register

The IF/ID pipeline register stores the outputs of the **Instruction Fetch (IF)** stage and provides them to the **Instruction Decode (ID)** stage in the next clock cycle. Pipeline registers act as state elements that hold intermediate values between pipeline stages and update their contents on the rising edge of the clock [1].

4.1.1 Inputs

- **clk** : Clock signal controlling when the register updates.
- **reset** : Clears the pipeline register during processor reset.
- **stall** : Prevents the register from updating when the pipeline is stalled.
- **flush** : Clears the register when incorrect instructions must be removed (e.g., branch taken).
- **instr_in [31:0]** : Instruction fetched from instruction memory.

- **pc_in [63:0]** : Program counter value of the fetched instruction.
- **pc4_in [63:0]** : Value of PC + 4.

4.1.2 Outputs

- **instr_out [31:0]** : Instruction forwarded to the decode stage.
- **pc_out [63:0]** : PC value forwarded to the next stage.
- **pc4_out [63:0]** : PC + 4 forwarded to the next stage.

4.1.3 Operation

The register updates on the rising edge of the clock.

- If **reset** or **flush** is asserted, all outputs are cleared to zero. This effectively inserts a bubble into the pipeline.
- If the pipeline is **not stalled**, the inputs from the fetch stage are stored and passed to the decode stage.
- If **stall** is asserted, the register holds its current values, preventing the pipeline from advancing.

Thus, the IF/ID register ensures correct instruction flow between the fetch and decode stages while supporting hazard handling mechanisms such as stalling and flushing.

4.2 ID/EX Pipeline Register

The ID/EX pipeline register stores the outputs of the **Instruction Decode (ID)** stage and forwards them to the **Execute (EX)** stage in the next clock cycle. It transfers operand values, immediate values, register identifiers, and control signals required by the ALU and later pipeline stages.

4.2.1 Inputs

Control signals

- **RegWrite_in** : Enables register write in WB stage
- **MemRead_in** : Enables memory read for load instructions
- **MemWrite_in** : Enables memory write for store instructions
- **MemtoReg_in** : Selects memory output for register writeback
- **Branch_in** : Indicates branch instruction
- **ALUSrc_in** : Selects immediate as ALU operand
- **ALUOp_in [1:0]** : ALU operation control

Data signals

- **pc4_in [63:0]** : PC + 4 value
- **rs1_data_in [63:0]** : Data read from register rs1
- **rs2_data_in [63:0]** : Data read from register rs2
- **imm_in [63:0]** : Sign-extended immediate value
- **branch_target_in [63:0]** : Calculated branch target address

Instruction fields

- **rs1_in, rs2_in, rd_in [4:0]** : Source and destination register numbers
- **funct3_in [2:0]** : ALU operation field
- **instr30_in** : Bit used for ALU control (e.g., ADD/SUB)

Additional control signals:

- **reset** : Clears the register
- **flush** : Removes incorrect instructions from pipeline
- **bubble** : Inserts a NOP into the pipeline

4.2.2 Outputs

All inputs are forwarded to corresponding outputs for use in the EX stage:

- Data values: **pc4_out, rs1_data_out, rs2_data_out, imm_out, branch_target_out**
- Register identifiers: **rs1_out, rs2_out, rd_out**
- Instruction fields: **funct3_out, instr30_out**
- Control signals: **RegWrite_out, MemRead_out, MemWrite_out, MemtoReg_out, Branch_out, ALUSrc_out, ALUOp_out**

4.2.3 Operation

The register updates on the rising edge of the clock.

- If **reset** or **flush** is asserted, all outputs are cleared.
- Otherwise, data and instruction fields are forwarded from the ID stage.
- If **bubble** is asserted, all control signals are set to zero while data values are still propagated. This inserts a NOP instruction into the pipeline.

Thus, the ID/EX register transfers decoded instruction information to the execute stage while supporting hazard handling through flush and bubble insertion.

4.3 EX/MEM Pipeline Register

The EX/MEM pipeline register stores the outputs of the **Execute (EX)** stage and forwards them to the **Memory (MEM)** stage in the next clock cycle. It holds the ALU computation results, branch evaluation signals, memory operation signals, and the destination register identifier.

4.3.1 Inputs

Data signals

- **alu_result_in [63:0]** : Result produced by the ALU
- **rs2_data_in [63:0]** : Value from register rs2 used for store instructions
- **branch_target_in [63:0]** : Calculated branch target address

Instruction fields

- **rd_in [4:0]** : Destination register number
- **zero_in** : Zero flag from ALU used for branch decision

Control signals

- **RegWrite_in** : Enables register write in WB stage
- **MemRead_in** : Enables memory read operation
- **MemWrite_in** : Enables memory write operation
- **MemtoReg_in** : Selects memory data for writeback
- **Branch_in** : Indicates branch instruction

4.3.2 Outputs

All input signals are stored and forwarded to the MEM stage:

- **alu_result_out**
- **rs2_data_out**
- **branch_target_out**
- **rd_out**
- **zero_out**
- **RegWrite_out**

- **MemRead_out**
- **MemWrite_out**
- **MemtoReg_out**
- **Branch_out**

4.3.3 Operation

The register updates on the rising edge of the clock.

- If **reset** is asserted, all outputs are cleared to zero.
- Otherwise, the inputs from the Execute stage are stored and forwarded to the Memory stage in the next cycle.

The ALU result is used either as a memory address (for load/store instructions) or as the final computation result for arithmetic instructions. The **zero** signal together with the **Branch** control signal is used to determine whether a branch is taken.

4.4 MEM/WB Pipeline Register

The MEM/WB pipeline register stores the outputs of the **Memory (MEM)** stage and forwards them to the **Write Back (WB)** stage in the next clock cycle. This register holds the data that will be written back to the register file.

4.4.1 Inputs

Data signals

- **mem_data_in [63:0]** : Data read from memory for load instructions
- **alu_result_in [63:0]** : ALU result forwarded from the EX stage

Instruction field

- **rd_in [4:0]** : Destination register number

Control signals

- **RegWrite_in** : Enables register write in the WB stage
- **MemtoReg_in** : Selects memory data instead of ALU result for write-back

4.4.2 Outputs

The stored signals are forwarded to the Write Back stage:

- **mem_data_out**
- **alu_result_out**
- **rd_out**
- **RegWrite_out**
- **MemtoReg_out**

4.4.3 Operation

The register updates on the rising edge of the clock.

- If **reset** is asserted, all outputs are cleared.
- Otherwise, the values from the MEM stage are stored and forwarded to the WB stage in the next cycle.

During the Write Back stage, the processor selects either the memory output or the ALU result based on the **MemtoReg** signal and writes the selected value into the destination register specified by **rd**.

5 Hazard Handling Mechanisms

5.1 Pipeline Hazards Overview

In a pipelined processor, multiple instructions are executed simultaneously in different stages of the pipeline. While this improves instruction throughput, it can also introduce situations where instructions interfere with each

other. These situations are known as **pipeline hazards**. Pipeline hazards may prevent the next instruction in the pipeline from executing during the designated clock cycle.

Pipeline hazards are generally classified into three categories [1]:

- **Structural Hazards**

  Structural hazards occur when multiple instructions attempt to use the same hardware resource at the same time. In this processor design, structural hazards are avoided by using separate instruction and data memories and by ensuring that each pipeline stage has its own dedicated hardware components.

- **Data Hazards**

  Data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed execution. For example, if an instruction needs a register value that is still being computed by an earlier instruction, the correct value may not yet be available.

  To handle data hazards, this processor implements a **forwarding unit** that forwards results from later pipeline stages to earlier stages. In cases where forwarding cannot resolve the dependency (such as load-use hazards), a **hazard detection unit** stalls the pipeline and inserts a bubble.

- **Control Hazards**

  Control hazards occur due to branch instructions that change the program counter. When a branch instruction is encountered, the processor may fetch subsequent instructions before the branch decision is resolved.

  In this processor, control hazards are handled using a **static branch prediction strategy** that assumes branches are not taken. If a branch is later determined to be taken, the incorrect instructions in the pipeline are flushed.

The following sections describe the mechanisms implemented in the processor to handle these hazards, including data forwarding, hazard detection, and pipeline flushing.

5.2 Data Forwarding Unit

Data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed execution. In a pipelined processor, this situation commonly arises when the result of an instruction is needed by the following instruction before it is written back to the register file.

To reduce pipeline stalls caused by such dependencies, the processor implements a **forwarding unit**. The forwarding unit bypasses results from later pipeline stages and directly forwards them to the ALU inputs in the Execute stage.

5.2.1 Inputs

The forwarding unit monitors register and control signals from different pipeline stages:

- **exmem_RegWrite** : Indicates that the EX/MEM stage will write to a register.
- **exmem_rd** [**4:0**] : Destination register in the EX/MEM stage.
- **memwb_RegWrite** : Indicates that the MEM/WB stage will write to a register.
- **memwb_rd** [**4:0**] : Destination register in the MEM/WB stage.
- **idex_rs1** [**4:0**] : Source register 1 of the instruction in the EX stage.
- **idex_rs2** [**4:0**] : Source register 2 of the instruction in the EX stage.

5.2.2 Outputs

- **forward_a** [**1:0**] : Select signal for the first ALU operand.
- **forward_b** [**1:0**] : Select signal for the second ALU operand.

These signals control multiplexers that determine whether the ALU operands should come from the register file or be forwarded from later pipeline stages.

5.2.3 Forwarding Logic

The forwarding logic checks whether the destination register of an instruction in a later pipeline stage matches the source registers of the current instruction in the EX stage.

EX Hazard (MEM → EX)

If the instruction in the EX/MEM stage writes to a register and its destination register matches the source register of the instruction in the EX stage, the ALU input is forwarded from the EX/MEM stage.

MEM Hazard (WB → EX)

If the required value is not available in the EX/MEM stage but is being written in the MEM/WB stage, the value is forwarded from the MEM/WB stage.

The forwarding control signals have the following meaning:

Signal Value	Source of ALU Operand
00	Register file (normal operation)
01	Forward from MEM/WB stage
10	Forward from EX/MEM stage

Additionally, the forwarding logic ignores register `x0` since it is always zero in the RISC-V architecture.

5.3 Hazard Detection Unit

Forwarding resolves most data hazards by supplying values from later pipeline stages to the ALU inputs. However, forwarding cannot resolve **load-use hazards**. This occurs when an instruction immediately following a load instruction requires the loaded value before it is available.

For example:

```
ld x1, 0(x2)
add x3, x1, x4
```

The result of the load instruction is only available after the MEM stage, so the following instruction cannot obtain the correct operand through forwarding alone. In such cases, the pipeline must be stalled for one cycle.

The **hazard detection unit** identifies this situation and generates a signal indicating a load-use hazard.

5.3.1 Inputs

- **idex_MemRead** : Indicates that the instruction in the EX stage is a load instruction.
- **idex_rd [4:0]** : Destination register of the instruction in the EX stage.
- **ifid_rs1 [4:0]** : Source register 1 of the instruction in the ID stage.
- **ifid_rs2 [4:0]** : Source register 2 of the instruction in the ID stage.
- **ifid_opcode [6:0]** : Opcode of the instruction in the ID stage.

5.3.2 Register Usage Detection

The unit first determines whether the instruction in the ID stage actually uses the `rs1` or `rs2` registers. This is done by checking the instruction opcode. For example:

- R-type instructions use both `rs1` and `rs2`.
- I-type arithmetic and load instructions use only `rs1`.
- Store and branch instructions use both registers.

Two internal signals, `rs1_used` and `rs2_used`, indicate whether each register operand is required by the instruction.

5.3.3 Hazard Condition

A load-use hazard is detected when:

- The instruction in the EX stage is a load instruction.
- The destination register of that instruction matches a source register of the instruction currently in the ID stage.
- The destination register is not register `x0`.

Formally, the hazard condition is:

$$idex_MemRead \wedge (idex_rd \neq 0) \wedge ((rs1_used \wedge idex_rd = ifid_rs1) \vee (rs2_used \wedge idex_rd = ifid_rs2))$$

When this condition is true, the output signal **load_use_hazard** is asserted.

5.3.4 Pipeline Response

When a load-use hazard is detected, the control logic stalls the pipeline and inserts a bubble into the next stage. This allows the load instruction to complete its memory access before the dependent instruction proceeds.

5.4 Control Hazards

Control hazards arise from branch instructions that change the program counter (PC). Since the processor fetches instructions sequentially, instructions following a branch may already enter the pipeline before the branch decision is resolved. If the branch is taken, these instructions are incorrect and must be removed from the pipeline.

In this processor, control hazards are handled through branch resolution in the Execute stage and a pipeline flush mechanism.

5.4.1 Branch Resolution

Branch instructions are resolved in the **Execute (EX)** stage. The ALU compares the source register operands and generates the `zero` signal. The branch decision is determined using the control signal `Branch` and the ALU comparison result.

The branch decision logic is implemented in the top-level processor module `pipe_processor.v`. The branch is taken when

$$branch_taken = Branch \wedge zero$$

where `Branch` indicates a branch instruction and `zero` is the result of the ALU comparison.

The branch target address is calculated earlier in the pipeline and propagated through the pipeline registers to the EX stage.

5.4.2 Program Counter Update

The next value of the program counter is selected between the sequential address and the branch target address. This logic is implemented in `pipe_processor.v`. The processor uses a static branch prediction strategy that assumes branches are not taken, so the PC normally increments to `PC + 4`. If the branch is taken, the PC is updated with the branch target address.

5.4.3 Pipeline Flush Mechanism

When a branch is taken, instructions that were fetched after the branch must be removed from the pipeline. This is achieved by flushing the pipeline registers.

The flush signal is generated in `pipe_processor.v` and is connected to the pipeline registers.

- The **IF/ID pipeline register** implemented in `if_id_reg.v` clears its stored instruction when the `flush` signal is asserted. This removes the incorrectly fetched instruction from the pipeline.
- The **ID/EX pipeline register** implemented in `id_ex_reg.v` also supports flushing, clearing its stored values to insert a bubble into the pipeline.

By clearing these pipeline registers, incorrect instructions are prevented from progressing further through the pipeline, ensuring correct execution when control flow changes due to a branch.

6 Processor Implementation

The complete pipelined processor is implemented in the top-level module `pipe_processor.v`. This module integrates all functional units, pipeline registers, and hazard handling mechanisms to form the final 5-stage pipelined RISC-V processor. The datapath is divided into the five standard pipeline stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Intermediate results are transferred between stages using pipeline registers.

6.1 Instruction Fetch Stage

The Instruction Fetch stage retrieves instructions from instruction memory using the current program counter value. The program counter is implemented in `pc.v`, which stores the current instruction address and updates it on every clock cycle unless the pipeline is stalled.

The module `pc_plus_4.v` computes the next sequential instruction address by adding four to the current PC value. Instruction memory is implemented in `instruction_mem.v`, which outputs the instruction corresponding to the address provided by the program counter.

The outputs of the fetch stage are stored in the IF/ID pipeline register implemented in `if_id_reg.v`. This register holds the fetched instruction and PC values so that the next instruction can be fetched while the current instruction moves to the decode stage.

6.2 Instruction Decode Stage

The Instruction Decode stage extracts the instruction fields and reads the required register operands. The register file is implemented in `reg_file.v`, which provides two read ports and one write port for register access.

The control signals for the instruction are generated by the module `control.v`. Based on the instruction opcode, this module generates signals such as `RegWrite`, `MemRead`, `MemWrite`, `MemtoReg`, `Branch`, and `ALUSrc`.

Immediate values used by instructions are generated using `imm_gen.v`. For branch instructions, the branch target address is calculated using the module `branch_targ_adder.v`. The outputs of this stage are stored in the ID/EX pipeline register implemented in `id_ex_reg.v`.

6.3 Execute Stage

The Execute stage performs arithmetic and logical operations using the ALU. The ALU operation is determined by the module `alu_control.v`, which generates the appropriate control signal based on the instruction fields and the ALU operation code.

The arithmetic and logical computations are performed by the ALU implemented in `alu.v`. Operand selection is performed using multiplexers implemented in `mux.v`, which select between register operands and immediate values depending on the instruction type.

Data hazards are resolved using the forwarding unit implemented in `forwarding_unit.v`. This unit forwards results from later pipeline stages to the ALU inputs when necessary. The outputs of the execute stage are stored in the EX/MEM pipeline register implemented in `ex_mem_reg.v`.

Branch decisions are also resolved in this stage using the ALU comparison result and the `Branch` control signal.

6.4 Memory Access Stage

The Memory stage performs data memory operations for load and store instructions. Data memory is implemented in `data_mem.v`. The memory address is provided by the ALU result from the previous stage, and data is either read from memory or written to memory depending on the control signals.

The results of the memory access stage are stored in the MEM/WB pipeline register implemented in `mem_wb_reg.v`.

6.5 Write Back Stage

In the Write Back stage, the processor writes results back to the register file. A multiplexer selects between the ALU result and the data read from memory based on the `MemtoReg` control signal. This multiplexer is implemented using the module `mux.v`. The selected value is written to the destination register in the register file.

6.6 Hazard Handling Integration

Hazard handling logic is integrated into the processor using dedicated modules. The forwarding logic is implemented in `forwarding_unit.v`, which resolves most data hazards by forwarding results from later pipeline stages.

Load-use hazards are detected by the module `hazard_detection_unit.v`. When such a hazard is detected, the pipeline is stalled and a bubble is inserted to delay execution until the required data becomes available.

Control hazards caused by branch instructions are handled in the execute stage. When a branch is taken, the pipeline registers are flushed and the program counter is updated with the branch target address.

6.7 Pipeline Integration

The integration of all these modules within `pipe_processor.v` forms the complete pipelined datapath. Each pipeline stage operates concurrently on different instructions, while pipeline registers maintain correct data flow between stages. Hazard detection, forwarding, and pipeline flushing ensure correct program execution while maintaining high instruction throughput.

6.8 Clock Cycle Methodology

The processor uses a synchronous clocking methodology to coordinate the operation of all pipeline stages. In this approach, sequential elements update their state based on clock edges, while combinational modules compute their outputs continuously based on their current inputs. This ensures that data moves through the pipeline in a controlled and predictable manner.

6.8.1 Positive Edge Triggered Pipeline Registers

Most state elements in the processor update their values on the rising edge of the clock signal. These elements store intermediate results between pipeline stages and allow multiple instructions to be processed simultaneously.

The following modules update on the positive edge of the clock:

- Program Counter (`pc.v`)
- IF/ID pipeline register (`if_id_reg.v`)
- ID/EX pipeline register (`id_ex_reg.v`)
- EX/MEM pipeline register (`ex_mem_reg.v`)
- MEM/WB pipeline register (`mem_wb_reg.v`)

Each of these modules contains an `always @(posedge clk)` block that captures input values and forwards them to the next pipeline stage in the following cycle. This mechanism ensures that all stages progress synchronously with the clock.

6.8.2 Combinational Logic Between Clock Edges

Between clock edges, combinational modules compute outputs based on their current inputs. These modules do not store state and therefore do not depend on the clock directly.

Examples of combinational modules in the processor include:

- Arithmetic Logic Unit (`alu.v`)
- ALU Control Unit (`alu_control.v`)

- Control Unit (`control.v`)
- Immediate Generator (`imm_gen.v`)
- Branch Target Adder (`branch_targ_adder.v`)
- Multiplexers (`mux.v`)
- Forwarding Unit (`forwarding_unit.v`)
- Hazard Detection Unit (`hazard_detection_unit.v`)

These components compute outputs within the same clock cycle, allowing the next pipeline stage to receive valid inputs when the next clock edge occurs.

6.8.3 Negative Edge Usage

Some memory or register file implementations use the negative clock edge for write operations. This design technique prevents conflicts between read and write operations occurring within the same clock cycle. By performing writes on the falling edge of the clock and reads on the rising edge, the processor ensures that newly written values become available for subsequent instructions without causing data hazards.

6.8.4 Pipeline Progression

During each clock cycle, instructions advance by one stage in the pipeline:

- Instruction Fetch retrieves the next instruction.
- Instruction Decode reads register operands and generates control signals.
- Execute performs ALU operations and branch comparisons.
- Memory stage accesses data memory for load and store instructions.
- Write Back updates the register file with the final result.

The pipeline registers ensure that results from each stage are stored at the clock edge and passed to the next stage in the following cycle. This allows multiple instructions to be processed concurrently, significantly improving processor throughput.

6.8.5 Reset Behavior

A global reset signal is used to initialize the processor to a known state. When the reset signal is asserted, the program counter and all pipeline registers are cleared. This prevents invalid instructions from being executed when the processor starts.

Specifically:

- The program counter in `pc.v` is reset to the initial instruction address.
- The pipeline registers (`if_id_reg.v`, `id_ex_reg.v`, `ex_mem_reg.v`, and `mem_wb_reg.v`) clear their stored values.
- Control signals in the pipeline are reset to zero, effectively inserting NOP instructions.

After reset is deasserted, the processor begins normal pipeline operation with instruction fetching starting from the initial program counter value.

7 Verification and Testing

7.1 Testbench Implementation

The functionality of the pipelined processor is verified using the testbench module `pipe_tb.v`. This testbench instantiates the processor and simulates its operation over multiple clock cycles while monitoring the internal state of the system.

7.1.1 Processor Instantiation

The processor under test is instantiated as the unit under test (UUT) using the module `pipe_processor`. The testbench provides the clock and reset signals required for processor operation.

```
pipe_processor uut (
    .clk(clk),
    .reset(reset)
);
```

7.1.2 Clock Generation

The testbench generates a periodic clock signal using a simple toggling mechanism:

```
always #5 clk = ~clk;
```

This creates a clock with a period of 10 ns, corresponding to a frequency of 100 MHz. The clock drives all sequential elements in the processor including the program counter and pipeline registers.

7.1.3 Reset Initialization

At the beginning of the simulation, the reset signal is asserted to initialize the processor to a known state. The reset signal remains active for the first 20 ns of the simulation.

```
reset = 1;
#20 reset = 0;
```

During this period, the program counter and pipeline registers are cleared, ensuring that no invalid instructions are executed when the processor begins operation.

7.1.4 Instruction Loading

The testbench reads instructions from the file `instructions.txt`. Each line in the file represents a single instruction encoded in hexadecimal format.

The testbench counts the number of instructions present in the file and uses this value to determine when the processor has reached the end of the instruction stream. The end of the file is treated as a halt condition for the processor.

7.1.5 Simulation Control

The processor is allowed to execute instructions until either the end of the instruction file is reached or a timeout condition occurs.

The simulation loop increments a cycle counter every clock cycle to track the number of cycles required to execute the program.

- If the program counter reaches the end of the instruction memory, the simulation terminates normally.
- If execution exceeds 2000 cycles, the testbench triggers a timeout condition to prevent infinite simulation.

7.1.6 Waveform Generation

The testbench generates simulation waveforms using the following commands:

```
$dumpfile("pipe_tb.vcd");
$dumpvars(0, pipe_tb);
```

These waveforms allow detailed inspection of processor behavior, including pipeline stage activity, control signals, and hazard handling.

7.1.7 Result Logging

After simulation completes, the contents of the processor register file are written to an output file named `register_file.txt`. The testbench iterates through all 32 registers and prints their values in hexadecimal format.

```
for (i = 0; i < 32; i = i + 1)
    $fdisplay(outfile, "%016h", uut.RF.regs[i]);
```

The total number of cycles required to execute the program is also written to the output file. This information is used to verify the correctness of the processor and evaluate its performance.

7.2 Test Cases and Expected Results

To verify the correctness of the pipelined processor, several test programs were executed using the testbench described earlier. Each test program targets a specific feature of the processor such as arithmetic operations, data forwarding, memory access, hazard detection, and control hazard handling.

The testbench reads instructions from `instructions.txt` and terminates execution when the program counter reaches the end of the instruction file (EOF). Since the testbench does not drain the pipeline, assume that additional dummy instructions are appended to the end of each program to allow earlier instructions to complete the write-back stage. This have not been shown in the examples.

7.2.1 Test Case 1: Arithmetic Operations

This test verifies the correct execution of arithmetic and logical instructions.

Instruction Sequence

```
addi x1, x0, 0
addi x2, x0, 5
addi x3, x0, 7
add  x4, x2, x3
sub  x5, x3, x2
and  x6, x2, x3
or   x7, x2, x3
```

Expected Register Values

Register	Value
x1	0
x2	5
x3	7
x4	12
x5	2
x6	5
x7	7

7.2.2 Test Case 2: Forwarding from EX Stage

This test verifies that the forwarding unit correctly resolves data hazards between dependent instructions.

Instruction Sequence

```
addi x1, x0, 5
addi x2, x0, 10
add  x3, x1, x2
sub  x4, x3, x1
```

The second arithmetic instruction depends on the result of the previous instruction. The forwarding unit forwards the result directly from the EX/MEM stage to the ALU input.

Expected Register Values

Register	Value
x1	5
x2	10
x3	15
x4	10

7.2.3 Test Case 3: Forwarding from MEM/WB Stage

This program tests forwarding when results must be forwarded from the MEM/WB pipeline stage.

Instruction Sequence

```
addi x1, x0, 3
addi x2, x0, 4
add  x3, x1, x2
add  x4, x3, x2
add  x5, x4, x1
```

Expected Register Values

Register	Value
x1	3
x2	4
x3	7
x4	11
x5	14

7.2.4 Test Case 4: Load-Use Hazard

This test verifies the hazard detection unit and pipeline stall mechanism.

Instruction Sequence

```
addi x1, x0, 8
addi x2, x0, 20
ld   x3, 0(x1)
add  x4, x3, x2
```

The instruction `add x4, x3, x2` depends on the result of the load instruction. The hazard detection unit inserts a stall cycle to allow the load instruction to complete.

Expected Register Values

Register	Value
x1	8
x2	20
x3	0
x4	20

7.2.5 Test Case 5: Branch Not Taken

This test verifies that the processor correctly continues sequential execution when a branch condition is false.

Instruction Sequence

```
addi x1, x0, 5
addi x2, x0, 7
beq  x1, x2, skip
addi x3, x0, 1
skip:
addi x4, x0, 2
```

Expected Register Values

Register	Value
x1	5
x2	7
x3	1
x4	2

7.2.6 Test Case 6: Branch Taken

This test verifies the pipeline flush mechanism when a branch is taken.

Instruction Sequence

```
addi x1, x0, 5
addi x2, x0, 5
beq  x1, x2, label
addi x3, x0, 99
label:
addi x4, x0, 7
```

The instruction writing to `x3` is flushed from the pipeline because the branch is taken.

Expected Register Values

Register	Value
x1	5
x2	5
x4	7

7.3 Waveform Analysis

The processor behavior was verified by analyzing the simulation waveforms generated from the VCD file (`pipe_tb.vcd`). The waveforms were viewed using Surfer to observe pipeline behavior, forwarding activity, hazard detection, and branch handling. The following figures show representative waveforms for each test case.

7.3.1 Test Case 1: Arithmetic Operations

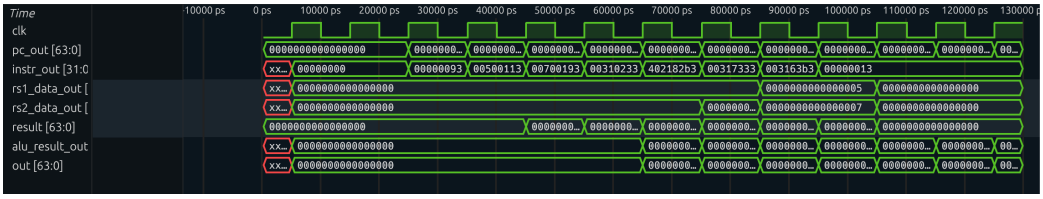


Figure 1: Waveform showing execution of arithmetic instructions. The signals illustrate the movement of instructions through the pipeline stages and the ALU producing the correct results before write-back.

7.3.2 Test Case 2: EX Stage Forwarding

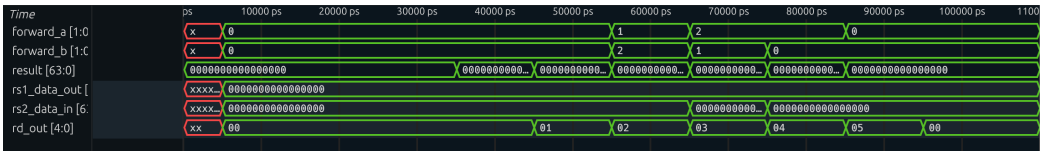


Figure 2: Waveform demonstrating forwarding from the EX/MEM pipeline stage. The forwarding control signals enable the ALU to use the result of a previous instruction without stalling the pipeline.

8 Team Contributions

Anish Toshniwal:
Pipeline Registers, Top Module, Report.
Subal Manchanda:
Forwarding Unit, Hazard Detection Unit, Report.
Parth Jigar Desai:
Control Hazard Handling, Top level tb, Report.

References

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: RISC-V Edition*. Morgan Kaufmann.